HeritageGraph Skills (`hg-skill`)

Engineering an Agentic, Ontology-Grounded Pipeline for Himalayan Cultural Heritage

Centre for Artificial Intelligence Research Nepal (CAIR-Nepal)

Architecture, Technical Implementation, and Verification Walkthrough

Table of Contents

1. **Part I — Introduction & Motivation** (Domain complexity, HeritageGraph, why agentic skills?)
2. **Part II — System Architecture & Run Lifecycle** (Orchestration, workspaces, manifest.json)
3. **Part III — Pipeline Walkthrough (Stages 00–04)**
    - Stage 00: Orchestrator & Execution Runtime
    - Stage 01: Ingestion, Preprocessing & LinkML Container Extraction
    - Stage 02: Entity Grounding, Multi-Authority Lookup & Review Queue
    - Stage 03: Deterministic RDF Triplification & YARRRML Export
    - Stage 04: Semantic Quality Gate & pySHACL Shape Validation
4. **Part IV — Technical Deep Dive** (LinkML, CIDOC-CRM, LUX/Wikidata/Getty, Graded SKOS)
5. **Part V — End-to-End Walkthrough** (Real dataset: Patan Durbar survey)
6. **Part VI — Quality Assurance & Test Verification** (84 automated tests, coverage, gates)
7. **Part VII — Current Status & Technical Roadmap**
8. **Part VIII — Conclusion & Strategic Vision**

Part I

Introduction & Motivation

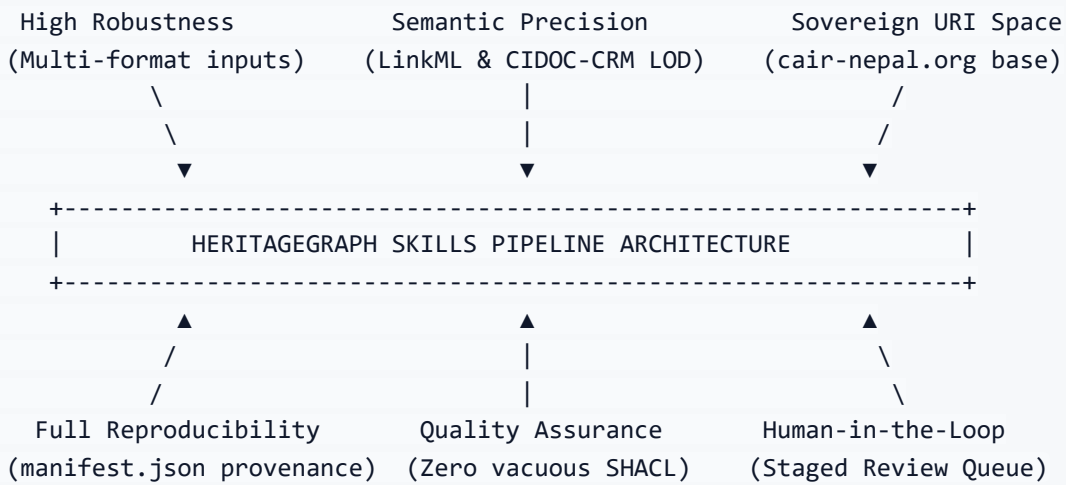
The Heritage Challenge: Himalayan Living Heritage

- **Heterogeneous Physical Records**: Archival survey reports (multi-page PDF), temple inventories (CSV/Excel), epigraphic inscriptions, and unstructured field journals.
- **Living Heritage Complexity**: Monuments in Nepal are active ritual hubs, not static monuments. They are tightly coupled to:
  - **Traditional Custodianship**: *Guthi* organizations (e.g. `SiGuthi`, `RajGuthi`, `SanaGuthi`) managing rituals and conservation.
  - **Socio-Religious Synchronization**: Syncretic worship (*Newar Buddhist-Hindu deities*, e.g. *Karunamaya / Rato Machhindranath*).
  - **Multiple Historical Eras**: Dates recorded in *Nepal Sambat* (NS), *Vikram Sambat* (VS), *Shaka Sambat* (ŚS), and Common Era (CE).
- **The Failure of Generic Extractors**: Off-the-shelf NLP or ungrounded LLM scrapers hallucinate entities, flatten relationships, destroy temporal semantics, and emit disconnected JSON rather than validated Linked Open Data (LOD).

What is HeritageGraph & `hg-skill`?

- **HeritageGraph Platform (`heritagegraph.ttl`)**:
  - An open semantic knowledge graph initiative by **CAIR-Nepal** (cairnepal.github.io/heritagegraphontology).
  - Models monuments, intangible festivals, architectural styles, Guthis, and structural interventions.
  - Canonical URI namespace: `https://cair-nepal.org/heritagegraph/` (`heritageGraph:`).
- **`hg-skill` Repository**:
  - The production transformation engine converting unstructured and tabular archives into validated RDF Turtle (`.ttl`).
  - Implemented as composable, modular agent skills under `skills/`:
    - Deterministic code where precision is required (schema compilation, SHACL validation, URI minting).
    - LLM/Agent-assisted capabilities where semantic inference is necessary (OCR text normalization, living heritage extraction).

Pipeline Design Goals



1. **Deterministic Guarantees**: Identical inputs and schemas produce bit-identical RDF graphs.
2. **Strict Validation Gates**: No invalid triple enters the global Knowledge Graph; graphs must satisfy pySHACL shapes.
3. **Traceability**: Every run records full SHA-256 digests, git commits, and tool execution logs.

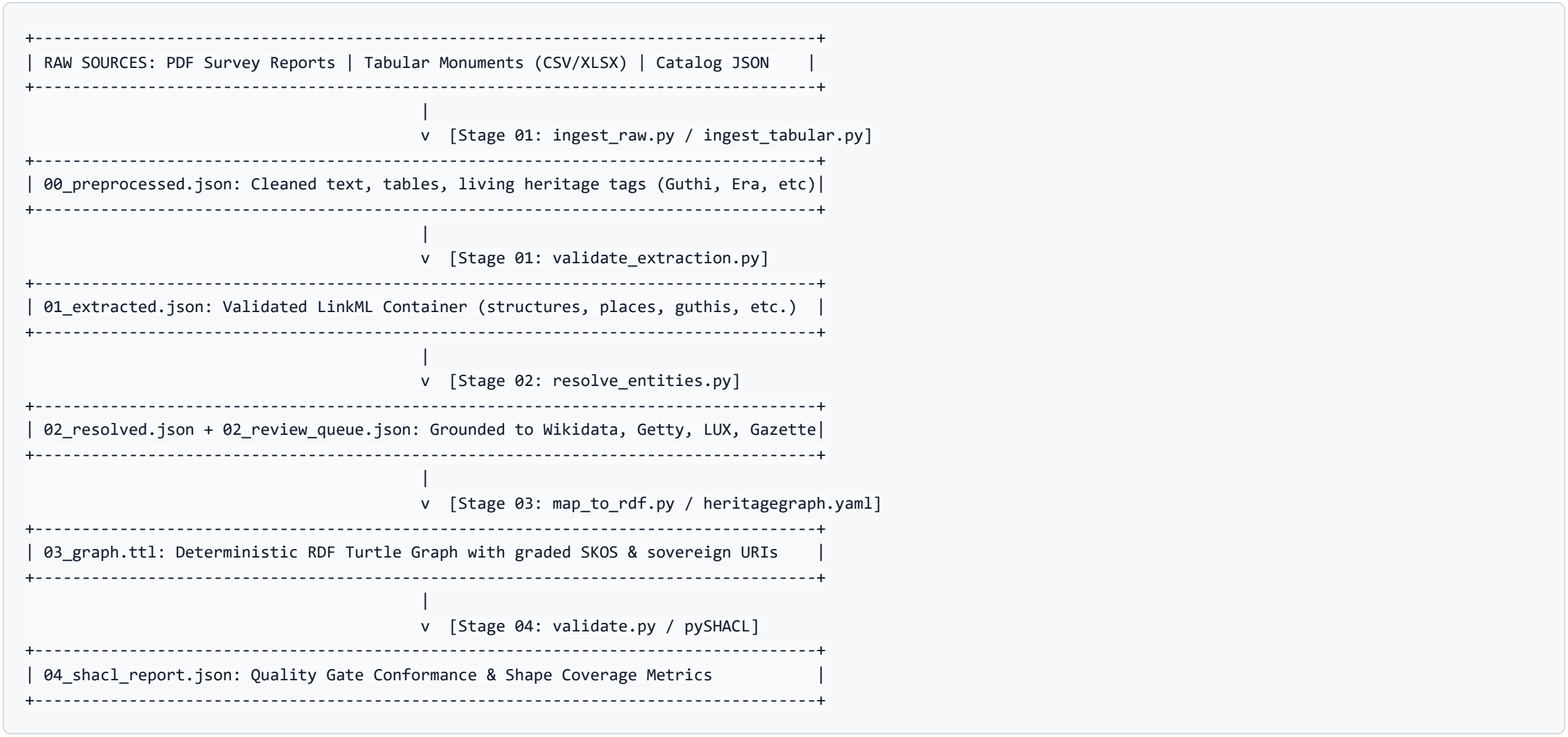
Part II

System Architecture & Repository Layout

Repository Organization

```
hg-skill/
├── schemas/                        # Authoritative upstream ontology definitions
│   ├── HeritageGraph.yaml          # LinkML source schema (v1.0.1)
│   ├── HeritageGraph.ttl           # Compiled OWL/RDFS ontology (4,244 triples)
│   ├── HeritageGraph.shacl.ttl     # Authoritative SHACL validation shapes
│   └── SOURCES.md                  # Upstream release tracking & sync instructions
├── skills/                         # Autonomous transformation stages
│   ├── 00_heritage-to-rdf/         # Pipeline runner & end-to-end orchestrator
│   ├── 01_heritage-extract/        # Multi-format ingestion, OCR repair & LinkML extraction
│   ├── 02_heritage-entity-resolution/ # Gazetteers, Wikidata, Getty, Yale LUX lookups
│   ├── 03_heritage-rdf-mapper/     # Deterministic triplifier & YARRRML generator
│   ├── 04_heritage-shacl-validate/# pySHACL conformance engine & coverage gate
│   └── common/                     # Shared RunWorkspace & manifest engine
├── runs/                           # Isolated execution workspaces (runs/<run_id>/)
├── tests/                          # 84 automated pytest integration & regression tests
└── docs/                           # Architecture Decision Records (ADRs) & specifications
```

End-to-End Execution Flow



RunWorkspace: Isolated Execution & Provenance

- **Module**: `skills/common/run_workspace.py` ([`RunWorkspace`](file:///c:/Users/nabin/Desktop/myall/hg-skill/skills/common/run_workspace.py))
- **Design Decision (ADR-0003)**: No shared mutable state. Every pipeline execution creates an isolated folder under `runs/<timestamp>_<slug>/`.
- **Standardized Stage Artifacts**:
  - `00_preprocessed.json`: Raw chunks and scanned candidate tags.
  - `01_extracted.json`: Schema-validated LinkML Container.
  - `02_resolved.json`: Reconciled entities with external URIs & `match_type`.
  - `02_review_queue.json`: Staged ambiguous or ungrounded items for human review.
  - `03_graph.ttl`: Triplified RDF Turtle knowledge graph.
  - `04_shacl_report.json`: Detailed SHACL conformance report.
  - `manifest.json`: Complete cryptographic provenance envelope.

Manifest-Driven Reproducibility (`manifest.json`)


```
{
  "run_id": "2026-10-25_idjmss-patan",
  "status": "completed",
  "source": {
    "file_name": "admin,+IDJMSS-1(1)_89-103.pdf",
    "byte_size": 143382,
    "sha256": "60c4dcc35da1680aa6ae00d2648b70697ea47b6759e98a146cfa09b31f2743e6"
  },
  "environment": {
    "schema_version": "1.0.1",
    "git_commit": "04fdc31",
    "python_version": "3.11.9"
  },
  "stages": {
    "stage_00_ingest": { "status": "success", "duration_seconds": 0.01 },
    "stage_01_extract": { "status": "success", "metrics": { "total_entities": 15 } },
    "stage_02_resolve": { "status": "success", "metrics": { "linked": 1, "local": 14 } },
    "stage_03_map": { "status": "success", "metrics": { "triples": 11, "mapping": "heritagegraph" } },
    "stage_04_validate": { "status": "success", "metrics": { "conforms": true, "violations": 0 } }
  }
}
```


Every run logs input hashes, git commit, duration, tool versions, and stage metrics.

Part III

Pipeline Walkthrough (Stages 00–04)

Stage 00: Pipeline Orchestrator

- **Skill Location**: `skills/00_heritage-to-rdf/`
- **Driver Script**: [`skills/00_heritage-to-rdf/scripts/run_pipeline.py`](file:///c:/Users/nabin/Desktop/myall/hg-skill/skills/00_heritage-to-rdf/scripts/run_pipeline.py)
- **Functionality**:
  - Dispatches stages in strict topological sequence: `01_extract` → `02_resolve` → `03_map` → `04_validate`.
  - Automatically coordinates inputs and outputs using `RunWorkspace`.
  - Supports interactive human-in-the-loop pass-through (`--interactive`).
  - Employs configurable ontology targets (`--ontology heritagegraph` vs `--ontology cidoc-crm`).
- **CLI Invocation**:

```
# Run stages 2 through 4 end-to-end on a staged extraction
python skills/00_heritage-to-rdf/scripts/run_pipeline.py --run-dir runs/<run_id>
```

Stage 01: Ingestion & Preprocessing

- **Problem**: Raw field reports and spreadsheets have messy OCR breaks, inconsistent headers, and unstandardized cultural terminology.
- **Scripts**:
  - `ingest_raw.py`: Format router dispatching to PDF, CSV, XLSX, or JSON parsers.
  - `ingest_pdf.py`: Uses `pypdf` to extract text, strip running headers/footers, and rejoin hyphenated line breaks.
  - `ingest_tabular.py`: Distinguishes structured administrative metadata from free-text architectural descriptions.
  - `recognize_heritage.py`: Regex and lexicon scanning for living heritage elements.
- **Output (`00_preprocessed.json`)**:
  - Normalized text chunks paired with pre-identified candidate tags for Guthis, Eras, and Architectural Styles.

Stage 01: Living Heritage Scanner

[`recognize_heritage.py`](file:///c:/Users/nabin/Desktop/myall/hg-skill/skills/01_heritage-extract/scripts/recognize_heritage.py) scans text against controlled vocabularies defined in LinkML enums:

Category	Recognized Patterns & Permissible Values	Ontology Mapping
Architectural Style	`Pagoda` , `Shikhara` , `Chaitya` , `Stupa` , `Dome`	`heritageGraph:hasArchitecturalStyle`
Guthi Types	`SiGuthi` , `RajGuthi` , `JatraGuthi` , `PujaGuthi` , `TempleGuthi` , `SanaGuthi`	`heritageGraph:Guthi`
Calendar Eras	`Nepal Sambat (NS)` , `Vikram Sambat (VS)` , `Shaka Sambat (ŚS)` , `CE/BCE`	`heritageGraph:CalendarSystem`
Religious Traditions	`Buddhist` , `Shaiva` , `Vaishnava` , `Shakta` , `Newar Syncretic`	`heritageGraph:hasReligiousTradition`

Prevents LLM extraction errors by seeding the extraction prompt with verified candidates.

Stage 01: LinkML Container Extraction

- **Script**: [`validate_extraction.py`](file:///c:/Users/nabin/Desktop/myall/hg-skill/skills/01_heritage-extract/scripts/validate_extraction.py)
- **Design Decision (ADR-0002)**: Intermediate extraction data MUST conform to the LinkML `Container` class in `schemas/HeritageGraph.yaml`.
- **Container Schema Collections**:

```yaml
Container:
  tree_root: true
  slots:
    - structures              # Architectural structures (monuments, shrines)
    - iconographic_objects    # Statues, murtis, inscriptions, chaityas
    - persons                 # Artisans, patrons, rulers, donors
    - guthis                  # Custodial guthi organizations
    - festivals               # Living cultural events and jatras
    - production_events       # Consecration and building events
    - places                  # Wards, toles, cities, complexes
```

- **Validation**: Validates required fields, enum boundaries, and slot ranges before writing `01_extracted.json`.

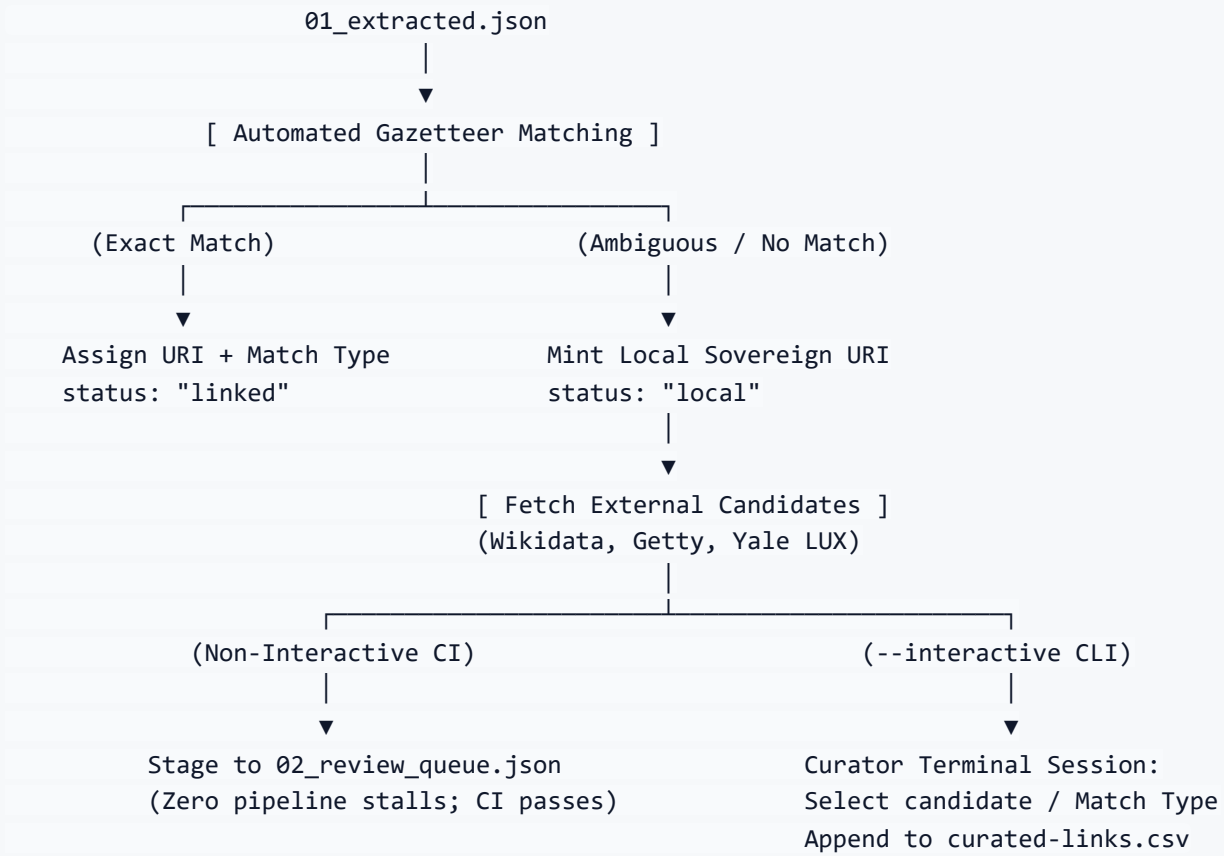
Stage 02: Entity Resolution & Grounding

- **Problem**: Kathmandu Valley heritage contains rampant homonyms (e.g. 50+ *Radha Krishna* or *Narayan* temples) and local institutions unindexed by global authorities.
- **Script**: [`resolve_entities.py`](file:///c:/Users/nabin/Desktop/myall/hg-skill/skills/02_heritage-entity-resolution/scripts/resolve_entities.py)
- **Multi-Authority Lookup**:
  - **Local Gazetteers** (`gazetteers/*.csv`): High-confidence internal index of local Newar places, toles, and historical monuments.
  - **Wikidata** (`lookup_wikidata.py`): Public search via `wbsearchentities` API.
  - **Getty Vocabularies** (`lookup_getty.py`): SPARQL queries against Getty AAT (concepts/materials), ULAN (artists), and TGN (places).
  - **Yale LUX** (`lookup_lux.py`): Linked Art REST queries for museum items, collections, concepts, and places.

Stage 02: Sovereign URIs & Graded SKOS

- **Sovereign URI Policy**:
  - HeritageGraph **never replaces an entity's primary subject URI** with an external authority URI.
  - Every entity receives a canonical HeritageGraph URI: `https://cair-nepal.org/heritagegraph/<kind>/<slug>`
  - External authorities are linked as outbound assertions.
- **Graded Mapping Semantics (Ticket #26)**:
  - Instead of flattening all links into `skos:exactMatch`, Stage 2 captures explicit semantic relationship degrees:
    - `exact` → `skos:exactMatch` (Identical concept)
    - `close` → `skos:closeMatch` (Similar/syncretic concept)
    - `broad` → `skos:broadMatch` (Specific style to general AAT term)
    - `narrow` → `skos:narrowMatch` (Broader category to specialized item)
    - `sameAs` → `owl:sameAs` (Co-referent physical individual)

Stage 02: Human-in-the-Loop Review Queue



Stage 03: Deterministic RDF Triplifier

- **Problem**: Triplification must be strictly deterministic, reproducible, and verifiable. Direct code hacks lead to schema drift.
- **Script**: [`map_to_rdf.py`](file:///c:/Users/nabin/Desktop/myall/hg-skill/skills/03_heritage-rdf-mapper/scripts/map_to_rdf.py)
- **Inputs**: `02_resolved.json` + Declarative Mapping (`heritagegraph.yaml`).
- **Core Characteristics**:
  - Pure Python execution using **RDFLib** (zero external dependencies).
  - Sub-second execution for hundreds of entities.
  - Emits typed RDF statements with bound prefixes (`heritageGraph:`, `crm:`, `skos:`, `owl:`, `prov:`).
  - Captures unmapped fields in execution metrics to prevent silent data loss.

Stage 03: Mapping Generation from LinkML

- **Generator Script**: [`generate_mapping.py`](file:///c:/Users/nabin/Desktop/myall/hg-skill/skills/03_heritage-rdf-mapper/scripts/generate_mapping.py)
- **Single Source of Truth**:
  - Instead of hand-authoring mapping rules, `generate_mapping.py` compiles `schemas/HeritageGraph.yaml` directly into:
    a. **Native Mapping**: `heritagegraph.yaml` (executed by `map_to_rdf.py`).
    b. **W3C YARRRML**: `heritagegraph.yarrrml.yaml` (exported for RML engines).
- **Hand-Written Overrides (`heritagegraph.overrides.yaml`)**:
  - Layers surgical exceptions over generated rules (e.g., mapping `note` to `rdfs:comment` or suppressing inherited slots).
- **CI Freshness Gate**:

```
python skills/03_heritage-rdf-mapper/scripts/generate_mapping.py --check
```

Exits with code 1 if committed mappings drift from the LinkML schema.

Stage 04: Quality Gate & pySHACL Validation

- **Problem**: Syntactically valid RDF can still violate semantic constraints (missing coordinates, untyped references, invalid calendar enums).
- **Script**: [`validate.py`](file:///c:/Users/nabin/Desktop/myall/hg-skill/skills/04_heritage-shacl-validate/scripts/validate.py)
- **Engine**: **pySHACL** validating `03_graph.ttl` against [`HeritageGraph.shacl.ttl`](file:///c:/Users/nabin/Desktop/myall/hg-skill/schemas/HeritageGraph.shacl.ttl).
- **Prevention of Vacuous Passes**:
  - A validator that tests an empty graph or a graph with incorrect namespace prefixes might report `CONFORMS: True` while checking zero shapes.
  - Stage 4 computes **per-shape coverage**. If zero shapes match the data graph, it marks the run as a **Vacuous Pass Failure** (`"vacuous": true`) and exits with code 1.

Part IV

Technical Deep Dive

LinkML Schema as Authoritative Backbone

- **File**: `schemas/HeritageGraph.yaml` (v1.0.1)
- LinkML (Linked Open Data Modeling Language) provides:
  i. Human-readable YAML object-oriented modeling with inheritance (`is_a`).
  ii. Single source of truth compiling into:
    - Python dataclasses.
    - JSON Schema for Stage 1 validation.
    - OWL / RDF Turtle ontology ([`HeritageGraph.ttl`](file:///c:/Users/nabin/Desktop/myall/hg-skill/schemas/HeritageGraph.ttl)).
    - SHACL validation shapes ([`HeritageGraph.shacl.ttl`](file:///c:/Users/nabin/Desktop/myall/hg-skill/schemas/HeritageGraph.shacl.ttl)).
    - Pipeline mapping configurations ([`heritagegraph.yaml`](file:///c:/Users/nabin/Desktop/myall/hg-skill/skills/03_heritage-rdf-mapper/assets/mappings/heritagegraph.yaml)).

Eliminates manual discrepancies across schema, validation, and serialization tiers.

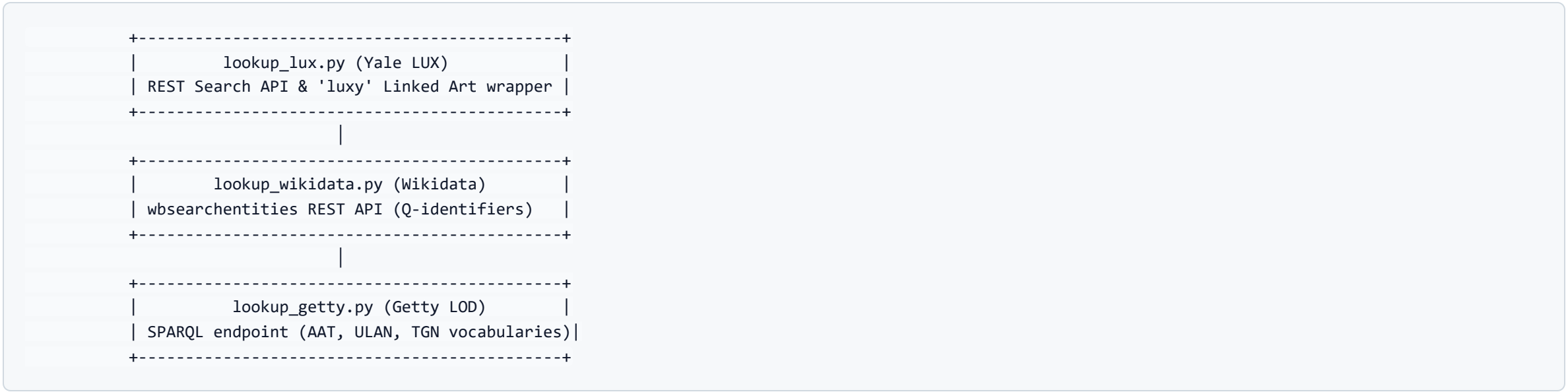
Dual-Track Ontology Modeling

`hg-skill` supports two distinct semantic representations:

Dimension	Native HeritageGraph (**`heritagegraph.yaml`**)	CIDOC-CRM (**`cidoc-crm.yaml`**)
Target Audience	Nepal Heritage researchers, SPARQL endpoints	International museum interoperability
Classes	`heritageGraph:ArchitecturalStructure` , `Guthi` , `Stupa`	`crm:E22_Human-Made_Object` , `E39_Actor`
Living Heritage	Direct properties (`managedByGuthi` , `enshrinesDeity`)	Reified event nodes (`E12_Production` , `E7_Activity`)
Shape Validation	Comprehensive (`HeritageGraph.shacl.ttl`)	Permissive subset (`--allow-crm-only`)
Pipeline Default	**Active Default** (ADR-0004)	Alternate export target

External LOD Connectivity: LUX, Wikidata, Getty

Implemented lookup adapters under `skills/02_heritage-entity-resolution/scripts/`:



Resilient design: Each client handles network timeouts, character encoding, and HTTP errors gracefully, falling back to local minting without crashing the pipeline.

YARRRML / RML Interoperability (ADR-0005)

- **Why not direct CSV-to-TTL via RML?**
  - Direct RML bypasses living heritage NER, diacritic folding, and human review.
  - Standard RML engines introduce heavy non-Python dependencies (Java JRE or Node.js).
- **The Solution (ADR-0005)**:
  - Keep the pure-Python pipeline (`map_to_rdf.py`) as the fast in-repo engine.
  - Generate standards-compliant **YARRRML** (`heritagegraph.yarrrml.yaml`) targeting `02_resolved.json` :

```
mappings:
  structures:
    sources:
      - ['02_resolved.json~jsonpath', '$.structures[*]']
    s: $(id)
    po:
      - [a, heritageGraph:ArchitecturalStructure]
      - [rdfs:label, $(name)]
      - [heritageGraph:managedByGuthi, $(managed_by_guthi)~iri]
      - [skos:exactMatch, $(resolution.uri)~iri]
```

Part V

Practical End-to-End Walkthrough

Transforming Patan Durbar Square Survey Data

Input: Raw Survey Report & Tabular Metadata

- **Real Dataset**: `runs/2026-10-25_idjmss-patan/` (from academic architectural survey of Patan Durbar Square monuments).
- **Raw Ingestion**:

```
python skills/01_heritage-extract/scripts/ingest_raw.py survey_patan.pdf
```

- **Preprocessed Text & Candidates** (`00_preprocessed.json`):
  - Normalized unicode text chunks.
  - Tagged candidates:
    - `"Krishna Mandir"` → Style: `Shikhara`
    - `"Char Narayan Mandir"` → Style: `Pagoda`
    - `"Bishwanath Mandir"` → Style: `Pagoda`
    - Chronology: `BCE`, `17th Century`, `2015 earthquake retrofitting`

Intermediate: Validated LinkML Container

From `runs/2026-10-25_idjmss-patan/01_extracted.json`:


```
{
  "id": "heritageGraph:batch-patan-durbar-survey-001",
  "structures": [
    {
      "id": "heritageGraph:struct-krishna-mandir",
      "name": "Krishna Mandir",
      "has_architectural_style": "Shikhara",
      "has_current_location": "heritageGraph:place-patan-durbar-square",
      "note": "Iconic multi-tiered stone temple dedicated to Krishna in Patan Durbar Square."
    },
    {
      "id": "heritageGraph:struct-sundari-chowk",
      "name": "Sundari Chowk",
      "has_current_location": "heritageGraph:place-patan-durbar-square",
      "note": "Royal palace courtyard renowned for master woodcarvings and stone architecture."
    }
  ]
}
```

Grounding: Entity Resolution (`02_resolved.json`)


```
{
  "id": "heritageGraph:struct-krishna-mandir",
  "name": "Krishna Mandir",
  "has_architectural_style": "Shikhara",
  "has_current_location": "heritageGraph:place-patan-durbar-square",
  "resolution": {
    "uri": "http://www.wikidata.org/entity/Q108882341",
    "status": "linked",
    "source": "wikidata",
    "match_type": "exact",
    "confidence": 0.95
  }
}
```


- For local entities without external match (e.g. *Manimandap Pati*), mints sovereign local URI: `https://cair-nepal.org/heritagegraph/structure/manimandap-pati` (`status: "local"`).
- Unresolved ambiguous entities stage candidates into `02_review_queue.json`.

Triplified Graph Output (`03_graph.ttl`)

```
@prefix heritageGraph: <https://cair-nepal.org/heritagegraph/> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix wikidata: <http://www.wikidata.org/entity/> .

heritageGraph:struct-krishna-mandir a heritageGraph:ArchitecturalStructure ;
    rdfs:label "Krishna Mandir" ;
    heritageGraph:hasArchitecturalStyle <https://cair-nepal.org/heritagegraph/ArchitecturalStyle#Shikhara> ;
    heritageGraph:hasCurrentLocation heritageGraph:place-patan-durbar-square ;
    rdfs:comment "Iconic multi-tiered stone temple dedicated to Krishna in Patan Durbar Square." ;
    skos:exactMatch wikidata:Q108882341 .

heritageGraph:struct-sundari-chowk a heritageGraph:ArchitecturalStructure ;
    rdfs:label "Sundari Chowk" ;
    heritageGraph:hasCurrentLocation heritageGraph:place-patan-durbar-square .
```

Note: Sovereign HeritageGraph URI is preserved as the subject; external Wikidata URI is linked via `skos:exactMatch`.

Quality Gate: Conformance & Coverage

From `runs/2026-10-25_idjmss-patan/04_shacl_report.json`:


```
{
  "conforms": true,
  "vacuous": false,
  "shape_coverage": {
    "matched": {
      "ArchitecturalStructure": 8,
      "Place": 3,
      "Person": 2
    },
    "total_shapes_evaluated": 15
  },
  "metrics": {
    "total_triples": 54,
    "violations_count": 0
  }
}
```


Verified: Non-vacuous pass; all structures conform to `HeritageGraph.shacl.ttl` constraints.

Part VI

Quality Assurance & Test Verification

Comprehensive Automated Test Suite

- **Test Framework**: `pytest`
- **Test Suite Metrics**: **84 tests passing across 16 test modules** (100% green).

```
tests/
├── test_interactive_resolution.py   (4 tests)  # Terminal HITL curation & gazetteer persistence
├── test_lookup_lux.py               (3 tests)  # Yale LUX API search & candidate normalization
├── test_pipeline_run_e2e.py         (2 tests)  # End-to-end lifecycle & native heritagegraph run
├── test_review_queue.py             (3 tests)  # Staging ambiguous items & candidate extraction
├── test_run_workspace.py            (8 tests)  # Run directory lifecycle, hashes & manifest.json
├── test_stage1_e2e.py               (5 tests)  # Raw ingestion to LinkML container extraction
├── test_stage1_pdf.py               (4 tests)  # OCR line repair & PDF text extraction
├── test_stage1_tabular.py           (6 tests)  # CSV/XLSX column routing & narrative splitting
├── test_stage1_scanner.py           (5 tests)  # Living heritage regex & enum recognition
├── test_stage3_container_mapping.py (8 tests)  # Deterministic triplification & prefix binding
├── test_stage3_entity_links.py      (6 tests)  # Graded SKOS matches & sovereign URI assertions
├── test_stage3_generate_mapping.py  (10 tests) # Schema mapping generation & freshness gates
└── test_stage4_coverage.py          (3 tests)  # SHACL validation & vacuous pass prevention
```

Key Test Guarantees

1. **Anti-Vacuity Guarantee** (`test_stage4_coverage.py`):
   - Confirms that Stage 4 explicitly fails runs where zero target nodes match SHACL shapes, preventing false-positive passes.
2. **Deterministic Freshness Gate** (`test_stage3_generate_mapping.py`):
   - Tests compare committed mappings (`heritagegraph.yaml`, `heritagegraph.yarrrml.yaml`) against the schema. If an engineer updates `HeritageGraph.yaml` without regenerating mappings, CI immediately goes red.
3. **Graded Relationship Integrity** (`test_stage3_entity_links.py`):
   - Verifies that `exact`, `close`, `broad`, `narrow`, and `sameAs` produce the precise RDF predicates without corrupting subject identifiers.
4. **Human Feedback Persistence** (`test_interactive_resolution.py`):
   - Verifies that curator decisions in interactive mode persist into `curated-links.csv` and reload automatically on subsequent runs.

Part VII

Current Implementation Status & Roadmap

Implementation Status Matrix

Capability	Status	Implemented Component	Notes
Multi-format Ingestion	**Production**	`ingest_raw.py` (PDF, CSV, XLSX, JSON)	OCR de-hyphenation, header cleanup
Living Heritage Scanner	**Production**	`recognize_heritage.py`	Guthis, Styles, Eras, Traditions
Container Validation	**Production**	`validate_extraction.py`	Schema-driven LinkML Container validation
Multi-Authority Lookup	**Production**	Wikidata, Getty, Yale LUX	Public APIs with resilient HTTP fallback
Graded SKOS & Identity	**Production**	`map_to_rdf.py`, `resolve_entities.py`	`exact`, `close`, `broad`, `narrow`, `sameAs`
Interactive HITL CLI	**Production**	`resolve_entities.py --interactive`	Curation loop + `curated-links.csv`
Provenance Tracking	**Production**	`skills/common/run_workspace.py`	SHA-256, git commit, `manifest.json`
Native RDF Mapping	**Production**	`map_to_rdf.py`, `generate_mapping.py`	Pure-Python RDFLib, compiled from YAML
YARRRML Export	**Production**	`generate_mapping.py --format yarrrml`	Full RML ecosystem interoperability
SHACL Quality Gate	**Production**	`skills/04_heritage-shacl-validate/`	pySHACL + per-shape coverage gate

Technical Roadmap & Next Horizons

1. **Epigraphic Sanskrit & Newar Script Recognition**:
   - Extend Stage 1 to support Inscription Text analysis with *Ranjana*, *Prachalit*, and *Devanagari* transliteration normalization.
2. **Automated Complex Disambiguation**:
   - Implement vector-based semantic similarity on monument descriptions to rank candidate matches in `fetch_candidates()` prior to human curation.
3. **Federated SPARQL Integration**:
   - Direct SPARQL endpoint synchronization with CAIR-Nepal's triple store (`https://cair-nepal.org/sparql`).
4. **Autonomous Ingestion Agent Teams**:
   - Multi-agent execution for scraping living heritage records from online archives (e.g. DANAM, Heidelberg CATS) into standardized `hg-skill` runs.

Part VIII

Conclusion & Strategic Vision

Summary of Architectural Contributions

- **Domain-Grounded AI**: Combines LLM semantic extraction with deterministic LinkML schemas, preventing hallucination while capturing intricate living heritage.
- **Semantic Sovereignty**: Honors international LOD standards (Wikidata, Getty, LUX, CIDOC-CRM) while asserting sovereign, stable URIs for Nepal's heritage assets.
- **Compounding Human Knowledge**: Every human curation decision in Stage 2 enriches `curated-links.csv`, making the pipeline smarter and faster on future datasets.
- **Reproducible Heritage Engineering**: Cryptographic provenance manifests (`manifest.json`) and anti-vacuous SHACL gates ensure data contributed to HeritageGraph is verified, defensible, and preservation-grade.

Thank You!

Questions & Discussion

HeritageGraph Skills (`hg-skill`)
Centre for Artificial Intelligence Research Nepal (CAIR-Nepal)
Repository: github.com/CAIRNepal/hg-skill
Ontology Documentation: cairnepal.github.io/heritagegraphontology